CMSC131: ArrayLists, Wrappers, and Iteration

Chapter 9

By Naren Ayinala

May 2026

Contents

1 Why Learn ArrayList After Arrays? **3**

2 Creating and Understanding an ArrayList **3**

2.1 Declaration, construction, and type arguments 3

2.2 Indices start at zero 3

3 The Essential ArrayList Methods **4**

3.1 Adding elements with add 4

3.2 Reading and replacing: get and set 4

3.3 Removing elements: remove 4

3.4 contains, indexOf, isEmpty, and clear 4

4 Arrays Versus ArrayLists **5**

5 Wrapper Classes: Why ArrayList<Integer> Exists **5**

6 Autoboxing and Unboxing **6**

6.1 Parsing text into numbers 6

7 The Difference Between == and equals for Wrappers **7**

8 Removing Integer Elements: An Important Overload Trap **7**

9 Traversing an ArrayList with Loops **7**

9.1 An indexed for loop 7

9.2 An enhanced for loop 8

10 Removing Elements While Traversing **8**

11 Iterator: Walking Through a List Safely **9**

12 ArrayLists in Memory: References and Aliasing **9**

13 Lists of Custom Objects **10**

14 Converting Between Arrays and ArrayLists **10**

15 Complete Example: Keeping Track of Scores **10**

16 Final Comparison of Common Operations **11**

1 Why Learn ArrayList After Arrays?

An array gives us a fixed number of slots. If we create `new int[3]`, there are three elements and the array cannot grow to four elements. This is useful when the number of elements is known. But imagine keeping a list of students who register for a course: students can join or leave, so we would prefer a collection that grows and shrinks as needed. That is the purpose of `ArrayList`.

An `ArrayList` is a class in `java.util` that stores an ordered, index-based collection. It automatically manages its storage when elements are added or removed. The programmer still controls which elements are stored and where they appear.

```
import java.util.ArrayList;

ArrayList<String> students = new ArrayList<>();
students.add("Ada");
students.add("Grace");
students.add("Alan");
System.out.println(students); // [Ada, Grace, Alan]
```

Notice the angle brackets: `ArrayList<String>` means this list is intended to hold `String` references. This is a *generic type*: the type argument tells Java what kinds of elements belong in the collection.

2 Creating and Understanding an ArrayList

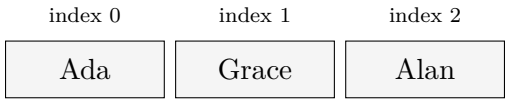
2.1 Declaration, construction, and type arguments

```
ArrayList<String> names; // Declare a reference variable.
names = new ArrayList<String>(); // Create an empty list.
ArrayList<String> cities = new ArrayList<>(); // Diamond shorthand.
```

The first line does not create a list. The constructor call with `new` creates the object. The empty diamond `<>` allows Java to infer the element type from the left-hand side.

2.2 Indices start at zero

The first element has index 0, just as with arrays. A list with three elements has valid indices 0, 1, and 2. The last valid index is `list.size() - 1`.



`size()` is a *method* and requires parentheses. An array's `length` is a field and has no parentheses.

```
int[] numbers = {3, 4, 5};
System.out.println(numbers.length); // 3
ArrayList<String> names = new ArrayList<>();
names.add("Ada");
System.out.println(names.size()); // 1
```

3 The Essential ArrayList Methods

3.1 Adding elements with add

`add(element)` appends an element to the end. `add(index, element)` inserts an element at the specified position, shifting the existing elements at that position and afterward to the right.

```
ArrayList<String> colors = new ArrayList<>();
colors.add("red"); // [red]
colors.add("blue"); // [red, blue]
colors.add(1, "green"); // [red, green, blue]
colors.add(0, "yellow"); // [yellow, red, green, blue]
```

For insertion, index `size()` is allowed and means append. An index greater than `size()` is invalid.

3.2 Reading and replacing: get and set

`get(index)` returns the element at the specified position. `set(index, value)` replaces the element at that position and returns the old value. Neither operation changes the list's size.

```
ArrayList<String> pets = new ArrayList<>();
pets.add("cat");
pets.add("dog");
System.out.println(pets.get(1)); // dog
String previous = pets.set(1, "bird");
System.out.println(previous); // dog
System.out.println(pets); // [cat, bird]
```

`set(2, "fish")` would be invalid here because index 2 does not exist. To create another element, use `add("fish")`.

3.3 Removing elements: remove

`remove(index)` removes an element and returns it; later elements shift left. `remove(object)` searches for a matching element and removes its first occurrence, returning a boolean indicating success.

```
ArrayList<String> words = new ArrayList<>();
words.add("sun");
words.add("moon");
words.add("sun");
String gone = words.remove(1); // removes moon
boolean found = words.remove("sun"); // removes first sun
System.out.println(words); // [sun]
System.out.println(gone); // moon
System.out.println(found); // true
```

3.4 contains, indexOf, isEmpty, and clear

```
ArrayList<String> foods = new ArrayList<>();
foods.add("rice");
foods.add("pasta");
System.out.println(foods.contains("rice")); // true
```

```
System.out.println(foods.indexOf("pasta")); // 1
System.out.println(foods.indexOf("soup")); // -1
System.out.println(foods.isEmpty()); // false
foods.clear();
System.out.println(foods.size()); // 0
```

These operations use the elements' equality behavior, normally `equals()`, rather than simply asking whether references point to the same object.

Method	Meaning
`add(x)`	Append an element
`add(i,x)`	Insert at position `i`
`get(i)`	Retrieve the element at position `i`
`set(i,x)`	Replace an existing element
`remove(i)`	Remove by index and return the removed element
`remove(x)`	Remove the first equal element; return success
`size()`	Number of elements
`contains(x)`	Whether an equal element exists
`indexOf(x)`	First matching index, or -1
`clear()`	Remove all elements

4 Arrays Versus ArrayLists

Feature	Array	ArrayList
Size	Fixed after creation	Can grow and shrink
Element access	`a[i]`	`list.get(i)`
Replacement	`a[i] = x`	`list.set(i,x)`
Number of elements	`a.length`	`list.size()`
Add/remove	Must handle manually	Built-in methods
Primitive elements	Allowed, e.g. `int[]`	Use wrapper types, e.g. `Integer`

```
int[] a = {10, 20, 30};
System.out.println(a[0]);
a[0] = 99;

ArrayList<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
System.out.println(list.get(0));
list.set(0, 99);
```

You cannot use `list[0]` or `list.length`; an ArrayList is an object with methods, not a Java array.

5 Wrapper Classes: Why ArrayList<Integer> Exists

`ArrayList` is generic, and generic type arguments must be reference types. Therefore `ArrayList<int>` is not legal. Java provides *wrapper classes* that represent primitive values as objects.

Primitive	Wrapper	Primitive	Wrapper
`byte`	`Byte`	`short`	`Short`
`int`	`Integer`	`long`	`Long`
`float`	`Float`	`double`	`Double`
`char`	`Character`	`boolean`	`Boolean`

```
// ArrayList<int> bad = new ArrayList<>(); // Compile error.
ArrayList<Integer> scores = new ArrayList<>();
scores.add(95);
scores.add(88);
ArrayList<Double> prices = new ArrayList<>();
prices.add(3.75);
ArrayList<Character> initials = new ArrayList<>();
initials.add('A');
```

The wrapper is an object; the primitive is not. This distinction matters for references, null values, and equality.

6 Autoboxing and Unboxing

Java automatically converts between primitives and their corresponding wrapper types in many contexts. Converting `int` to `Integer` is *autoboxing*; converting `Integer` to `int` is *unboxing*.

```
Integer boxed = 12; // Autoboxing from int.
int plain = boxed; // Unboxing to int.
ArrayList<Integer> nums = new ArrayList<>();
nums.add(7); // int is boxed into Integer.
int first = nums.get(0); // Integer is unboxed to int.
```

A wrapper reference may be `null`; a primitive cannot be `null`. Attempting to unbox null throws a `NullPointerException`.

```
Integer number = null;
// int value = number; // Compiles, but throws at runtime.
```

6.1 Parsing text into numbers

A `String` containing digits is still text. The wrapper classes provide parsing methods to produce primitive values.

```
int age = Integer.parseInt("19");
double rate = Double.parseDouble("3.5");
boolean active = Boolean.parseBoolean("true");
System.out.println(age + 1); // 20
```

`Integer.parseInt("nineteen")` throws `NumberFormatException`. The method `Integer.valueOf("19")` instead returns an `Integer` object.

7 The Difference Between == and equals for Wrappers

The `==` operator compares primitive values when both operands are primitive. When both operands are wrapper references, `==` compares reference identity. Java caches some wrapper values, so two wrappers can sometimes share an object. This makes `==` especially misleading.

```
Integer a = Integer.valueOf(1000);
Integer b = Integer.valueOf(1000);
System.out.println(a == b); // false for this example
System.out.println(a.equals(b)); // true
```

For ordinary value comparison of non-null wrappers, use `equals()`, or unbox deliberately when appropriate. Java may unbox when comparing a wrapper with a primitive, which changes what `==` means in that expression.

```
Integer wrapped = 42;
int primitive = 42;
System.out.println(wrapped == primitive); // true: unboxing
```

`equals()` also considers wrapper types: an `Integer` containing 3 does not equal a `Double` containing 3.0.

8 Removing Integer Elements: An Important Overload Trap

An `ArrayList<Integer>` has two `remove` methods. `remove(int index)` removes by index, while `remove(Object value)` removes a matching element. An integer literal selects the index overload.

```
ArrayList<Integer> numbers = new ArrayList<>();
numbers.add(10);
numbers.add(20);
numbers.add(30);
numbers.remove(1); // Removes element at index 1: 20.
System.out.println(numbers); // [10, 30]
numbers.remove(Integer.valueOf(10)); // Removes the value 10.
System.out.println(numbers); // [30]
```

Do not assume `numbers.remove(10)` removes the number 10: it attempts to remove index 10 and can throw `IndexOutOfBoundsException`.

9 Traversing an ArrayList with Loops

9.1 An indexed for loop

An indexed loop is useful when you need both an element and its position.

```
ArrayList<String> names = new ArrayList<>();
names.add("Ada"); names.add("Grace"); names.add("Alan");
for (int i = 0; i < names.size(); i++) {
    System.out.println(i + ": " + names.get(i));
}
```

The condition uses `i < names.size()`, not `i <= names.size()`. The latter eventually attempts an invalid index.

9.2 An enhanced for loop

The enhanced `for` loop visits each element in order without exposing an index variable.

```
for (String name : names) {
    System.out.println(name);
}
```

Changing the loop variable does not replace an element in the list:

```
for (String name : names) {
    name = "unknown"; // Reassigns only local variable.
}
System.out.println(names); // Original names unchanged.
```

To replace elements, use an indexed loop and `set()`.

```
for (int i = 0; i < names.size(); i++) {
    names.set(i, names.get(i).toUpperCase());
}
```

10 Removing Elements While Traversing

When an element is removed, later elements shift one position left. A forward loop that increments immediately after removal can skip an element.

```
ArrayList<Integer> nums = new ArrayList<>();
nums.add(2); nums.add(2); nums.add(3);
for (int i = 0; i < nums.size(); i++) {
    if (nums.get(i) == 2) {
        nums.remove(i);
    }
}
System.out.println(nums); // [2, 3] -- one 2 was skipped!
```

After removing index 0, the second 2 shifts to index 0, but the loop moves to index 1. One solution is to iterate backward:

```
for (int i = nums.size() - 1; i >= 0; i--) {
    if (nums.get(i) == 2) {
        nums.remove(i);
    }
}
```

Another solution is to decrement `i` after removal in a forward indexed loop. For ordinary enhanced-for traversal, do not structurally modify the list directly: it can trigger `ConcurrentModificationException`. Use an `Iterator` when removal during traversal is required.

11 Iterator: Walking Through a List Safely

An `Iterator` is an object that visits elements one by one. It exposes `hasNext()` to ask whether another element exists, and `next()` to retrieve it.

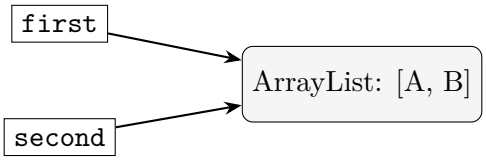
```
import java.util.ArrayList;
import java.util.Iterator;
ArrayList<String> items = new ArrayList<>();
items.add("keep");
items.add("remove");
items.add("keep again");
Iterator<String> it = items.iterator();
while (it.hasNext()) {
    String item = it.next();
    if (item.equals("remove")) {
        it.remove(); // Removes the last element returned by next().
    }
}
System.out.println(items); // [keep, keep again]
```

The iterator's own `remove()` method is designed to coordinate removal with traversal. Calling `it.remove()` before `next()`, or twice after one `next()`, is illegal.

12 ArrayLists in Memory: References and Aliasing

An ArrayList is an object. The variable `first` holds a reference to that object, not the collection itself. Assignment of one ArrayList variable to another copies the reference, not the elements.

```
ArrayList<String> first = new ArrayList<>();
first.add("A");
ArrayList<String> second = first;
second.add("B");
System.out.println(first); // [A, B]
```



To create a separate list with the same element references, use a copy constructor:

```
ArrayList<String> copy = new ArrayList<>(first);
copy.add("C");
System.out.println(first); // [A, B]
System.out.println(copy); // [A, B, C]
```

This is a *shallow copy*: the new list object has separate slots, but if its elements are mutable objects, the two lists may still refer to the same element objects.

13 Lists of Custom Objects

Suppose we have a `Car` class. An `ArrayList<Car>` stores references to `Car` objects, not copies of those objects.

```
class Car {
    String model;
    Car(String model) { this.model = model; }
}

ArrayList<Car> cars = new ArrayList<>();
Car car = new Car("Civic");
cars.add(car);
car.model = "Accord";
System.out.println(cars.get(0).model); // Accord
```

Because the list and `car` refer to the same `Car`, changing its field is visible through both references. A list of objects may also contain `null` unless your program prevents it.

14 Converting Between Arrays and ArrayLists

`Arrays.asList(...)` provides a fixed-size list backed by an array. It is not a normal resizable ArrayList, so `add()` and `remove()` are unsupported on that result. Wrap it in a new `ArrayList` when you need resizing.

```
import java.util.Arrays;
ArrayList<String> words = new ArrayList<>(
    Arrays.asList("one", "two", "three"));
words.add("four"); // Works: this is a resizable ArrayList.
String[] array = words.toArray(new String[0]);
System.out.println(Arrays.toString(array));
```

Be careful with primitive arrays: `Arrays.asList(new int[]1,2)` does not create a `List<Integer>` of two integers. Primitive arrays and generic collections have different element-type rules.

15 Complete Example: Keeping Track of Scores

This program reads and stores several scores, calculates the average, and prints the scores above that average. It combines wrapper classes, indexing, enhanced loops, and basic methods.

```
import java.util.ArrayList;

public class ScoreTracker {
    public static void main(String[] args) {
        ArrayList<Integer> scores = new ArrayList<>();
        scores.add(80);
        scores.add(95);
        scores.add(75);
        scores.add(90);

        int total = 0;
        for (int score : scores) {
            total += score; // Each Integer is unboxed to int.
```

```
        }
        double average = (double) total / scores.size();
        System.out.println("Average: " + average); // 85.0

        for (int i = 0; i < scores.size(); i++) {
            if (scores.get(i) > average) {
                System.out.println("Above average: " + scores.get(i));
            }
        }
    }
}
```

We first add four Integer elements. During the enhanced loop, each element is unboxed to an `int` value. The cast to `double` ensures the average is computed with floating-point division. Finally, an indexed loop visits each element and prints the values 95 and 90 because they are greater than 85.0.

16 Final Comparison of Common Operations

Task	Array	ArrayList
First element	`a[0]`	`list.get(0)`
Last element	`a[a.length-1]`	`list.get(list.size()-1)`
Replace first	`a[0] = x`	`list.set(0,x)`
Number of elements	`a.length`	`list.size()`
Print contents	`Arrays.toString(a)`	`System.out.println(list)`
Make separate container	`a.clone()`	`new ArrayList<>(list)`